Midterm: Part 3

Name: Joan Daniel Rivas Andrade

Introduction

This project is based on the kernel module developed in Workshop 3, called sysinfo_chardev, which creates the character device /dev/sysinfo. In its original version, the device returned general system information such as the kernel release, the value of jiffies, and the number of CPUs when the user executed cat /dev/sysinfo.

In this new version, the module was modified so that the user can control which information is returned by writing commands to the device from user space. The module now supports the commands all, kernel, cpu, and time, allowing the output to be selected dynamically.

Objective

The main objective of this project was to extend the sysinfo_chardev kernel module by adding support for command-based control through the device file /dev/sysinfo. This required implementing a write() operation so that the user could send commands from user space and select the information that would later be returned by read().

The specific goals were to support the commands all, kernel, cpu, and time, to verify the module behavior through Linux commands, and to justify why a character device driver is the correct choice for this implementation.

The project was developed and tested in the OS laboratory kernel environment. The module was built and executed in Linux using the 6.19.0-os2-lab kernel. Before starting the implementation, the kernel version and the required build tools were verified to make sure the environment was ready for external kernel module compilation.

The commands used to verify the environment included uname -r, gcc --version, and make --version.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ uname -r
gcc --version
make --version
6.19.0-os2-lab
gcc (Debian 15.2.0-12) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU Make 4.4.1
Este programa fue construido para x86_64-pc-linux-gnu
Copyright (C) 1988-2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$
```

Module Design and Implementation

The module was implemented as a Linux character device using register_chrdev() and a file_operations structure. The driver defines the operations open(), read(), and write().

The open() function is used to log access to the device. The write() function receives a command from user space, copies it into kernel space with copy_from_user(), removes newline characters, validates the command, and stores the selected mode internally. The valid commands are all, kernel, cpu, and time.

The read() function checks the current mode and generates the appropriate output. Depending on the last valid command received, it returns either all available information or only the specific item requested by the user. The output is copied back to user space with copy_to_user().

daniel@daniel: ~/Escritorio/sysinfo_project

Session Acciones Editar Vista Ayuda

GNU nano 8.7 sysinfo_chardev.c

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/init.h>
#include <linux/jiffies.h>
#include <linux/smp.h>
#include <linux/utsname.h>
#include <linux/string.h>

#define DEVICE_NAME "sysinfo"
#define BUF_LEN 256
#define CMD_LEN 32

static int major;
static char current_command[CMD_LEN] = "all";

enum sysinfo_mode {
    MODE_ALL,
    MODE_KERNEL,
    MODE_CPU,
    MODE_TIME
};

static enum sysinfo_mode current_mode = MODE_ALL;

static int dev_open(struct inode *inode, struct file *file)
{
    pr_info("sysinfo: device opened\n");
    return 0;
}

static ssize_t dev_write(struct file *file, const char __user *buffer, size_t len, loff_t *offset)
{
    char kbuf[CMD_LEN];
    size_t to_copy;

    to_copy = min(len, (size_t)(CMD_LEN - 1));

    if (copy_from_user(kbuf, buffer, to_copy))
        return -EFAULT;

    kbuf[to_copy] = '\0';

    strreplace(kbuf, '\n', '\0');
    strreplace(kbuf, '\r', '\0');

    if (strcmp(kbuf, "all") == 0) {
        current_mode = MODE_ALL;
        strscpy(current_command, "all", CMD_LEN);
    } else if (strcmp(kbuf, "kernel") == 0) {
        current_mode = MODE_KERNEL;
        strscpy(current_command, "kernel", CMD_LEN);
    } else if (strcmp(kbuf, "cpu") == 0) {
        current_mode = MODE_CPU;
        strscpy(current_command, "cpu", CMD_LEN);
    } else if (strcmp(kbuf, "time") == 0) {
```

[147 lineas leidas]

^G Ayuda ^O Guardar ^F Buscar ^K Cortar ^T Ejecutar ^C Ubicación M-U Deshacer
^X Salir ^R Leer fich. ^\ Reemplazar ^U Pegar ^J Justificar ^/ Ir a linea M-E Rehacer

```c
static ssize_t dev_read(struct file *file, char __user *buffer, size_t len, loff_t *offset)
{
    char msg[BUF_LEN];
    int msg_len;

    if (*offset > 0)
        return 0;

    switch (current_mode) {
        case MODE_KERNEL:
            msg_len = scnprintf(msg, BUF_LEN,
                                "Kernel release: %s\n",
                                init_uts_ns.name.release);
            break;

        case MODE_CPU:
            msg_len = scnprintf(msg, BUF_LEN,
                                "Number of CPUs: %u\n",
                                num_online_cpus());
            break;

        case MODE_TIME:
            msg_len = scnprintf(msg, BUF_LEN,
                                "Jiffies: %lu\n",
                                jiffies);
            break;

        case MODE_ALL:
        default:
            msg_len = scnprintf(msg, BUF_LEN,
                                "Kernel release: %s\n"
                                "Jiffies: %lu\n"
                                "Number of CPUs: %u\n",
                                init_uts_ns.name.release,
                                jiffies,
                                num_online_cpus());
            break;
    }
```

Compilation Process

The module was compiled as an out-of-tree kernel module using a Makefile configured to build against the currently running kernel. After the source file and the Makefile were created, the command make was executed in the project directory.

The compilation process generated the file sysinfo_chardev.ko, which is the loadable kernel module used for testing. The successful compilation confirmed that the source code was compatible with the selected kernel environment.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ make
make -C /lib/modules/6.19.0-os2-lab/build M=/home/daniel/Escritorio/sysinfo_project modules
make[1]: se entra en el directorio '/home/daniel/Escritorio/kernel/linux-6.19'
make[2]: se entra en el directorio '/home/daniel/Escritorio/sysinfo_project'
  CC [M]  sysinfo_chardev.o
  MODPOST Module.symvers
  CC [M]  sysinfo_chardev.mod.o
  CC [M]  .module-common.o
  LD [M]  sysinfo_chardev.ko
  BTF [M] sysinfo_chardev.ko
make[2]: se sale del directorio '/home/daniel/Escritorio/sysinfo_project'
make[1]: se sale del directorio '/home/daniel/Escritorio/kernel/linux-6.19'
```

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ ls -l sysinfo_chardev.ko
-rw-rw-r-- 1 daniel daniel 352720 mar 17 13:19 sysinfo_chardev.ko
```

Module Loading

After compilation, the module was loaded into the kernel with insmod. Once loaded, the system assigned a dynamic major number to the character device. This number was later required to create the device node manually in /dev.

The correct loading of the module was verified with lsmod, dmesg, and cat /proc/devices | grep sysinfo.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ sudo insmod sysinfo_chardev.ko
lsmod | grep sysinfo_chardev
dmesg | tail -20
cat /proc/devices | grep sysinfo
[sudo] contraseña para daniel:
Lo siento, pruebe otra vez.
[sudo] contraseña para daniel:
sysinfo_chardev        16384  0
[11685.629730] pcieport 0000:00:1d.1: AER: Correctable error message received from 0000:00:1d.1
[11685.629758] pcieport 0000:00:1d.1: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Receiver ID)
[11685.629764] pcieport 0000:00:1d.1:   device [8086:9d19] error status/mask=00000001/00002000
[11685.629772] pcieport 0000:00:1d.1:    [ 0] RxErr                  (First)
[11750.642098] pcieport 0000:00:1d.1: AER: Correctable error message received from 0000:00:1d.1
[11750.642117] pcieport 0000:00:1d.1: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Receiver ID)
[11750.642124] pcieport 0000:00:1d.1:   device [8086:9d19] error status/mask=00000001/00002000
[11750.642132] pcieport 0000:00:1d.1:    [ 0] RxErr                  (First)
[11814.192082] pcieport 0000:00:1d.1: AER: Correctable error message received from 0000:00:1d.1
[11814.192101] pcieport 0000:00:1d.1: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Receiver ID)
[11814.192108] pcieport 0000:00:1d.1:   device [8086:9d19] error status/mask=00000001/00002000
[11814.192115] pcieport 0000:00:1d.1:    [ 0] RxErr                  (First)
[11829.628874] pcieport 0000:00:1d.1: AER: Correctable error message received from 0000:00:1d.1
[11829.628900] pcieport 0000:00:1d.1: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Receiver ID)
[11829.628906] pcieport 0000:00:1d.1:   device [8086:9d19] error status/mask=00000001/00002000
[11829.628913] pcieport 0000:00:1d.1:    [ 0] RxErr                  (First)
[11910.422797] sysinfo_chardev: loading out-of-tree module taints kernel.
[11910.422801] sysinfo_chardev: module verification failed: signature and/or required key missing - tainting kernel
[11910.424853] sysinfo: registered with major number 234
[11910.424856] sysinfo: default command is 'all'
234 sysinfo
```

Device Creation and Permissions

Since the device node is not created automatically when using register_chrdev(), it was necessary to create /dev/sysinfo manually using mknod. The major number obtained from /proc/devices was used together with minor number 0.

After creating the node, permissions were changed with chmod 666 so that the device could be read from and written to during testing without permission problems.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ sudo mknod /dev/sysinfo c 234 0
sudo chmod 666 /dev/sysinfo
ls -l /dev/sysinfo
crw-rw-rw- 1 root root 234, 0 mar 17 13:21 /dev/sysinfo
```

Default Behavior

Before receiving any command from user space, the module uses all as the default mode. Therefore, when the user reads from /dev/sysinfo for the first time, the device returns the kernel release, the jiffies value, and the number of CPUs.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ cat /dev/sysinfo
Kernel release: 6.19.0-os2-lab
Jiffies: 4298343310
Number of CPUs: 4

┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$
```

Command all

The command all configures the device to return all supported system information. After writing this command to /dev/sysinfo, reading from the device returns the kernel release, the current jiffies value, and the number of CPUs.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ echo all | sudo tee /dev/sysinfo > /dev/null
cat /dev/sysinfo
Kernel release: 6.19.0-os2-lab
Jiffies: 4298394943
Number of CPUs: 4
```

Command kernel

The command kernel selects only the kernel release information. After writing this command to the device, the next read operation returns only the kernel version string.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ echo kernel | sudo tee /dev/sysinfo > /dev/null
cat /dev/sysinfo
Kernel release: 6.19.0-os2-lab
```

Command cpu

The command cpu configures the device to return only the number of CPUs currently available in the system. This confirms that the module can selectively expose one specific item of information.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ echo cpu | sudo tee /dev/sysinfo > /dev/null
cat /dev/sysinfo
Number of CPUs: 4
```

Command time

The command time configures the device to return only the current jiffies value. This test confirms that the driver can dynamically switch its output according to the last valid command received from user space.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ echo time | sudo tee /dev/sysinfo > /dev/null
cat /dev/sysinfo
Jiffies: 4298436603
```

Invalid Command Handling

The module was also tested with an invalid command to verify input validation. When an unsupported string was written to /dev/sysinfo, the write() function rejected it and returned -EINVAL. In user space, this appeared as an invalid argument error, while the kernel log recorded the invalid command.

This behavior confirms that the module correctly validates user input and avoids changing its internal mode when an unsupported command is received.

```
┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$ echo hola | sudo tee /dev/sysinfo > /dev/null
dmesg | tail -20
tee: /dev/sysinfo: Argumento inválido
[14121.620064] pcieport 0000:00:1d.1: PCIe Bus Error: severity=Correctable, type=Data Link Layer, (Receiver ID)
[14121.620071] pcieport 0000:00:1d.1:   device [8086:9d19] error status/mask=00000080/00002000
[14121.620080] pcieport 0000:00:1d.1:    [ 7] BadDLLP
[14131.916748] sysinfo: device opened
[14131.916770] sysinfo: command set to cpu
[14131.922252] sysinfo: device opened
[14169.619769] pcieport 0000:00:1d.1: AER: Correctable error message received from 0000:00:1d.1
[14169.619805] pcieport 0000:00:1d.1: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Receiver ID)
[14169.619815] pcieport 0000:00:1d.1:   device [8086:9d19] error status/mask=00000001/00002000
[14169.619827] pcieport 0000:00:1d.1:    [ 0] RxErr                  (First)
[14176.909056] sysinfo: device opened
[14176.909063] sysinfo: command set to time
[14176.910787] sysinfo: device opened
[14203.624938] pcieport 0000:00:1d.1: AER: Multiple Correctable error message received from 0000:00:1d.1
[14203.624976] pcieport 0000:00:1d.1: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Receiver ID)
[14203.624987] pcieport 0000:00:1d.1:   device [8086:9d19] error status/mask=00000041/00002000
[14203.625000] pcieport 0000:00:1d.1:    [ 0] RxErr                  (First)
[14203.625011] pcieport 0000:00:1d.1:    [ 6] BadTLP
[14232.360100] sysinfo: device opened
[14232.360122] sysinfo: invalid command: hola

┌──(daniel㉿daniel)-[~/Escritorio/sysinfo_project]
└─$
```

Why a Character Device is Appropriate

A character device driver is appropriate for implementing /dev/sysinfo because the device does not represent a storage medium and does not operate on fixed-size blocks of data. Instead, it provides information to the user as a sequential stream of bytes when standard read and write operations are performed.

The purpose of /dev/sysinfo is to exchange commands and system information directly with the kernel. This matches the behavior of a character device, which is intended for simple byte-oriented communication. A block device, on the other hand, is designed for storage devices such as disks and partitions, where data is managed in blocks. Since /dev/sysinfo does not store persistent data and does not require block-based access, a block device would not be appropriate.

Discussion

One important implementation detail was handling the newline character automatically added by the echo command. This issue was solved by removing newline and carriage return characters before comparing the input against the valid commands.

Another important point was command validation. By rejecting unsupported commands with -EINVAL, the module avoids undefined behavior and preserves a valid internal state. The use of copy_from_user() and copy_to_user() was also necessary to safely transfer data between user space and kernel space.

Finally, the use of the file offset in the read() function ensured that the device behaved correctly during read operations and did not repeatedly return the same data in a single access.

Conclusion

The sysinfo_chardev module was successfully extended to support command-based output control from user space. The implementation of the write() function allowed the user to select the desired information dynamically, while the read() function returned the correct output according to the chosen mode.

The tests confirmed that the commands all, kernel, cpu, and time work correctly and that invalid commands are properly rejected. This project demonstrates the usefulness of Linux character device drivers for controlled interaction between user space and kernel space.